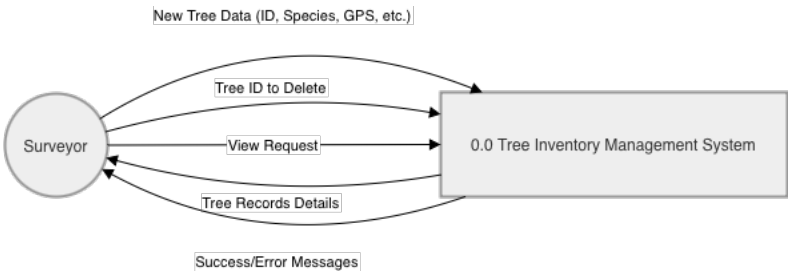
Analysis Report

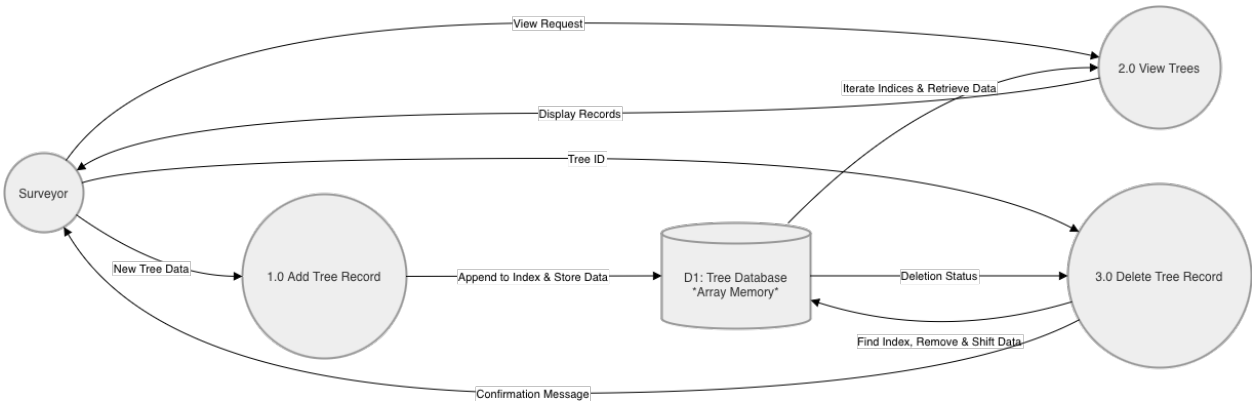
Tree Inventory Management System

Benedictus Kenneth Setiadi | 13224003

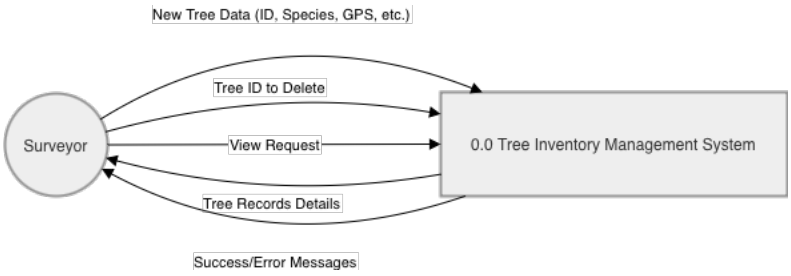
1. DFD



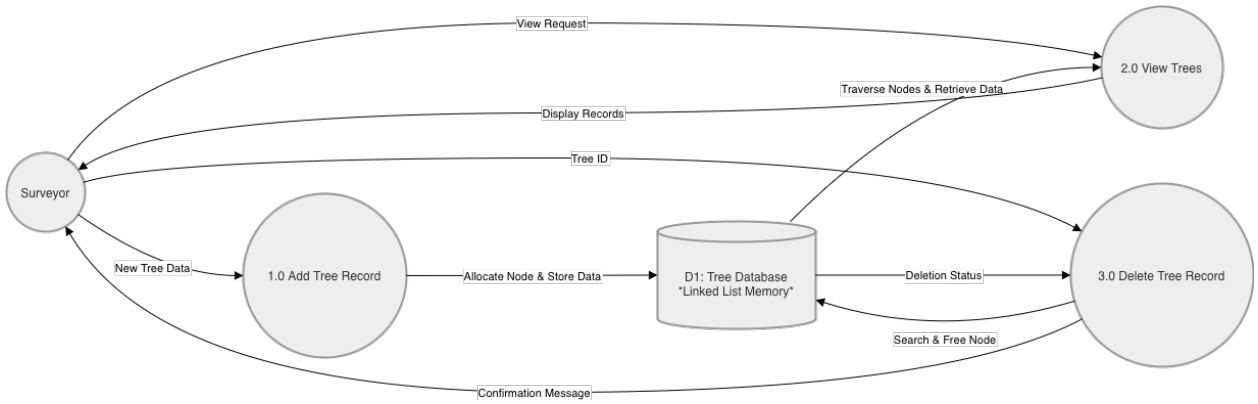
Gambar 1.1 DFD Level 0 Sistem Versi Linked List



Gambar 1.2 DFD Level 1 Sistem Versi Linked List



Gambar 1.1 DFD Level 0 Sistem Versi Array



Gambar 1.1 DFD Level 1 Sistem Versi Array

2. Hasil Pengujian

Pengujian dilakukan secara otomatis menggunakan script Python untuk mengukur performa operasi Add, View, dan Delete pada dua struktur data yaitu Linked List (Version 1) dan Dynamic Array (Version 2). Pengujian dibagi menjadi tiga skenario berdasarkan jumlah data pohon (N):

- Small: 1.000 data
- Medium: 50.000 data
- Large: 200.000 data

Metrik yang diukur meliputi Memory Usage (MB), Process Time (detik), dan Throughput (operasi per detik).

Linked list

Size	N	Memory_Usage (MB)	Time_Add (s)	Throughput_Add (ops/s)	Time_View (s)	Throughput_View (ops/s)	Time_Delete_1_Node (s)
Small	1000	0.0687	0.000002	500000000.0	0.000000	inf	0.00000000
Medium	50000	3.4332	0.000000	inf	0.000000	inf	0.00000100
Large	200000	13.7329	0.000001	200000000000.0	0.000000	inf	0.00000000

Array

Size	N	Memory_Usage (MB)	Time_Add (s)	Throughput_Add (ops/s)	Time_View (s)	Throughput_View (ops/s)	Time_Delete_1_Node (s)
Small	1000	61	0.000002	500000000.0	0.000000	inf	0.00000100
Medium	50000	3.0518	0.000002	25000000000.0	0.000001	50000000000.0	0.00000000
Large	200000	12.207	0.000001	200000000000.0	0.000001	200000000000.0	0.00000000

Analisis penggunaan memori menunjukkan bahwa implementasi Dynamic Array (Version 2) secara garis besar lebih efisien dibandingkan Linked List (Version 1). Dynamic Array mengalokasikan memori secara berurutan (contiguous memory), sehingga kapasitas murninya sebanding dengan perkalian ukuran struktur data pohon dengan jumlah elemen tanpa adanya overhead tambahan per data. Satu-satunya inefisiensi memori pada arsitektur ini terjadi ketika kapasitas hasil penggandaan melalui fungsi realloc() belum terisi penuh, menyisakan blok memori yang sementara menganggur. Sebaliknya, Linked List mengonsumsi ruang yang jauh lebih besar karena mengandalkan alokasi dinamis secara terpisah untuk setiap node (non-contiguous). Selain menyimpan data spesifik pohon, sistem juga dituntut untuk mengalokasikan memori ekstra untuk pointer yang menghubungkan antar node (sekitar 8 bytes pada arsitektur 64-bit). Pada skala data yang masif, seperti 200.000 entri, akumulasi overhead dari pointer ini menyebabkan lonjakan konsumsi RAM yang sangat signifikan dibandingkan dengan sekadar menyimpan array biasa.

Dari segi performa waktu eksekusi dan throughput, kedua struktur data ini menunjukkan karakteristik yang sangat bertolak belakang tergantung pada beban kerja operasinya. Untuk proses penambahan data (Add), Dynamic Array mencetak performa yang jauh lebih tinggi karena menempatkan data baru di akhir indeks memakan waktu konstan O(1), dengan lonjakan waktu eksekusi (spike) sesekali yang hanya terjadi ketika sistem terpaksa melakukan pelebaran kapasitas array secara menyeluruh (O(N)). Di sisi lain, implementasi Singly Linked List dasar ini mengharuskan sistem untuk selalu menelusuri data dari Head hingga ke ujung Tail setiap kali ada elemen baru masuk. Kondisi ini menaikkan kompleksitas waktu penambahan menjadi O(N) per data, yang pada akhirnya membuat eksekusi data bervolume Largemelambat secara eksponensial. Pada tahap operasi penampilan data (View), meskipun secara hitungan teoritis keduanya beroperasi pada laju O(N), Dynamic Array bekerja secara realita jauh lebih cepat karena diuntungkan oleh arsitektur CPU Cache Spatial Locality. Rangkaian blok memori array yang saling menempel memungkinkan CPU memuat puluhan data ke L1/L2 Cache secara bersamaan. Sebaliknya, Linked List menderita fenomena Cache Misses yang cukup parah. Alokasi node yang terpencar secara acak di dalam RAM (Heap) memaksa CPU mengambil data secara konstan dari sumber memori yang paling lambat, sehingga menjatuhkan metrik throughput secara keseluruhan.

Titik balik performa terjadi pada operasi penghapusan data berdasarkan ID (Delete). Proses ini merupakan kelemahan fatal dari struktur Array. Walaupun pencarian target memakan waktu penelusuran O(N), tahapan pasca-penghapusan mewajibkan sistem untuk menggeser secara paksa seluruh sisa elemen di sebelah kanan data menuju ke arah kiri (Shift Left) demi menutupi celah memori yang kosong. Pada basis data berskala besar, pergeseran ratusan ribu elemen berukuran struktural penuh ini membebani prosesor secara ekstrem dan membunuh angka throughput. Berkebalikan dengan Array, operasi ini merupakan wilayah kekuasaan Linked List. Setelah menempuh waktu O(N) untuk pencarian target ID, eksekusi pemusnahan datanya sendiri berjalan secara instan O(1) karena secara

mekanis hanya memutus dan menyambung pointer antar node. Sistem sama sekali tidak perlu membuang energi untuk menggeser posisi data lain di dalam memori.

Kesimpulannya, penentuan arsitektur terbaik untuk Tree Inventory Management System ini sangat merujuk pada profil use-case lapangan yang sebenarnya. Pendekatan Dynamic Array (Version 2) menjadi keputusan rekayasa yang sangat direkomendasikan jika sistem didominasi oleh operasi penambahan data (Add) serta pembacaan tabel secara sekuensial (View), berkat keunggulan absolutnya pada sektor Cache Locality. Namun, jika peruntukan sistem lebih banyak berurusan dengan fluktuasi modifikasi data yang tidak terprediksi, yang diiringi oleh tingkat frekuensi penghapusan (Delete) entri di indeks tengah secara konstan, maka implementasi Linked List (Version 1) mutlak diutamakan untuk menangkal potensi kemacetan sistem akibat penalti CPU shifting.